

torch.linalg.lstsq#

<https://pytorch.org/docs/stable/generated/torch.linalg.lstsq.html>

Description:

Computes a solution to the least squares problem of a system of linear equations. Letting $K \in \mathbb{R}$ or $\mathbb{C}$, the least squares problem for a linear system $AX=B$ with $A \in K^{m \times n}, B \in K^{m \times k}$ is defined as where $\|A\|_F$ denotes the Frobenius norm. Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions. driver chooses the backend function that will be used. For CPU inputs the valid values are 'gels', 'gelsy', 'gelsd', 'gelss'. To choose the best driver on CPU consider:

Function Signature

```
torch.linalg.lstsq(A, B, rcond=None, *, driver=None)#
```

Parameters

Keyword Arguments

Returns

Parameters

Keyword

`driver` (str, optional) – name of the LAPACK/MAGMA method to be used. If None, ‘gelsy’ is used for CPU inputs and ‘gels’ for CUDA inputs. Default: None.

Returns:

A named tuple (solution, residuals, rank, singular_values).

Code Examples

```
>>> A = torch.randn(1,3,3)
>>> A
tensor([[[[-1.0838,  0.0225,  0.2275],
          [ 0.2438,  0.3844,  0.5499],
          [ 0.1175, -0.9102,  2.0870]]]])
>>> B = torch.randn(2,3,3)
>>> B
tensor([[[[-0.6772,  0.7758,  0.5109],
          [-1.4382,  1.3769,  1.1818],
          [-0.3450,  0.0806,  0.3967]],
        [[[-1.3994, -0.1521, -0.1473],
          [ 1.9194,  1.0458,  0.6705],
          [-1.1802, -0.9796,  1.4086]]]])
>>> X = torch.linalg.lstsq(A, B).solution # A is broadcasted to
shape (2, 3, 3)
>>> torch.dist(X, torch.linalg.pinv(A) @ B)
tensor(1.5152e-06)

>>> S = torch.linalg.lstsq(A, B, driver='gelsd').singular_values
>>> torch.dist(S, torch.linalg.svdvals(A))
tensor(2.3842e-07)

>>> A[:, 0].zero_() # Decrease the rank of A
>>> rank = torch.linalg.lstsq(A, B).rank
>>> rank
tensor([2])
```

Notes & Warnings

NOTE

Note This function computes $X = A.\text{pinverse}() @ B$ in a faster and more numerically stable way than performing the computations separately.

⚠ WARNING

Warning The default value of `rcond` may change in a future PyTorch release. It is therefore recommended to use a fixed value to avoid potential breaking changes.

Detailed Documentation

Documentation

Computes a solution to the least squares problem of a system of linear equations.

Letting $K \in \{\mathbb{R}, \mathbb{C}\}$, the least squares problem for a linear system $AX=B$ with $A \in K^{m \times n}, B \in K^{m \times k}$ is defined as

where $\| \cdot \|_F$ denotes the Frobenius norm.

Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

driver chooses the backend function that will be used. For CPU inputs the valid values

are 'gels', 'gelsy', 'gelsd', 'gelss'. To choose the best driver on CPU consider:

If A is well-conditioned (its condition number is not too large), or you do not mind some precision loss.

For a general matrix: 'gelsy' (QR with pivoting) (default)

If A is full-rank: 'gels' (QR)

If A is not well-conditioned.

'gelsd' (tridiagonal reduction and SVD)

But if you run into memory issues: 'gelss' (full SVD).

For CUDA input, the only valid driver is 'gels', which assumes that A is full-rank.

See also the full description of these drivers

rcond is used to determine the effective rank of the matrices in A when driver is one of ('gelsy', 'gelsd', 'gelss'). In this case, if σ_i are the singular values of A in decreasing order, σ_i will be rounded down to zero if $\sigma_i \leq \text{rcond} \cdot \sigma_1$. If $\text{rcond} = \text{None}$ (default), rcond is set to the machine precision of the dtype of A times $\max(m, n)$.

This function returns the solution to the problem and some extra information in a named tuple of four tensors (solution, residuals, rank, singular_values). For inputs A, B of shape (*, m, n), (*, m, k) respectively, it contains

solution: the least squares solution. It has shape (*, n, k).

residuals: the squared residuals of the solutions, that is, $\|AX - B\|_F^2 / \|AX - B\|_F^2$. It has shape (*, k). It is computed when $m > n$ and every matrix in A

is full-rank, otherwise, it is an empty tensor. If A is a batch of matrices and any matrix in the batch is not full rank, then an empty tensor is returned. This behavior may change in a future PyTorch release.

rank: tensor of ranks of the matrices in A. It has shape equal to the batch dimensions of A. It is computed when driver is one of ('gelsy', 'gelsd', 'gelss'), otherwise it is an empty tensor.

singular_values: tensor of singular values of the matrices in A. It has shape (*, min(m, n)). It is computed when driver is one of ('gelsd', 'gelss'), otherwise it is an empty tensor.

This function computes $X = A.\text{pinverse}() @ B$ in a faster and more numerically stable way than performing the computations separately.

The default value of rcond may change in a future PyTorch release. It is therefore recommended to use a fixed value to avoid potential breaking changes.

A (Tensor) – lhs tensor of shape (*, m, n) where * is zero or more batch dimensions.

B (Tensor) – rhs tensor of shape (*, m, k) where * is zero or more batch dimensions.

rcond (float, optional) – used to determine the effective rank of A. If rcond= None, rcond is set to the machine precision of the dtype of A times max(m, n). Default: None.

driver (str, optional) – name of the LAPACK/MAGMA method to be used. If None, 'gelsy' is used for CPU inputs and 'gels' for CUDA inputs. Default: None.

A named tuple (solution, residuals, rank, singular_values).

